

Internship Task Submission Report

C/C++ Programming Track — Core Task Evaluation

Developer Profile: Harsha1692006

Compiler Tools: Apple Clang / GCC toolchain (g++)

Repository Link: <https://github.com/Harsha1692006/InternSpark-Tasks>

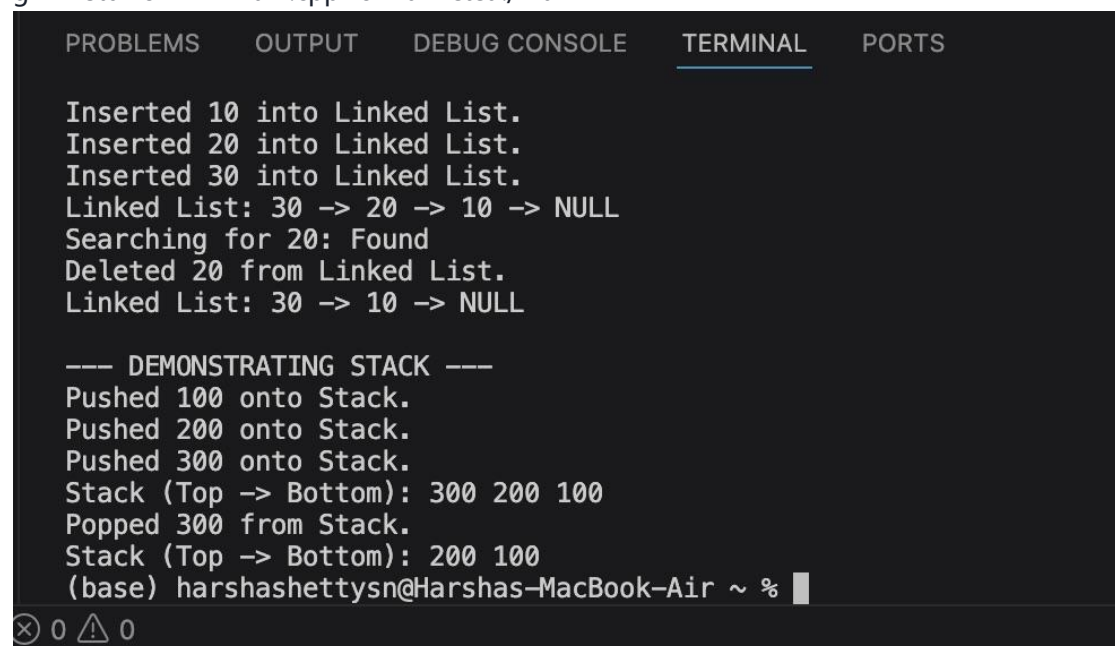
Task 1: Data Structures Implementation

Goal: Implement core fundamental data structures from scratch using pure C/C++ memory management,

avoiding external libraries or built-in collection primitives.

- **Selected Implementations:** Singly Linked List & Pointer-based Stack.
- **Key Methods Included:** Head insertion, value-targeted pointer node removals, linear element lookups, and explicit stack push/pop mutations.
- **Time Complexities Documented:** Insertion/Push: $O(1)$, Search/Remove: $O(n)$.

g++ -std=c++17 main.cpp -o main && ./main



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Inserted 10 into Linked List.
Inserted 20 into Linked List.
Inserted 30 into Linked List.
Linked List: 30 -> 20 -> 10 -> NULL
Searching for 20: Found
Deleted 20 from Linked List.
Linked List: 30 -> 10 -> NULL

--- DEMONSTRATING STACK ---
Pushed 100 onto Stack.
Pushed 200 onto Stack.
Pushed 300 onto Stack.
Stack (Top -> Bottom): 300 200 100
Popped 300 from Stack.
Stack (Top -> Bottom): 200 100
(base) harshashettysn@Harshas-MacBook-Air ~ %
```

Task 2: Algorithms & Problem Solving

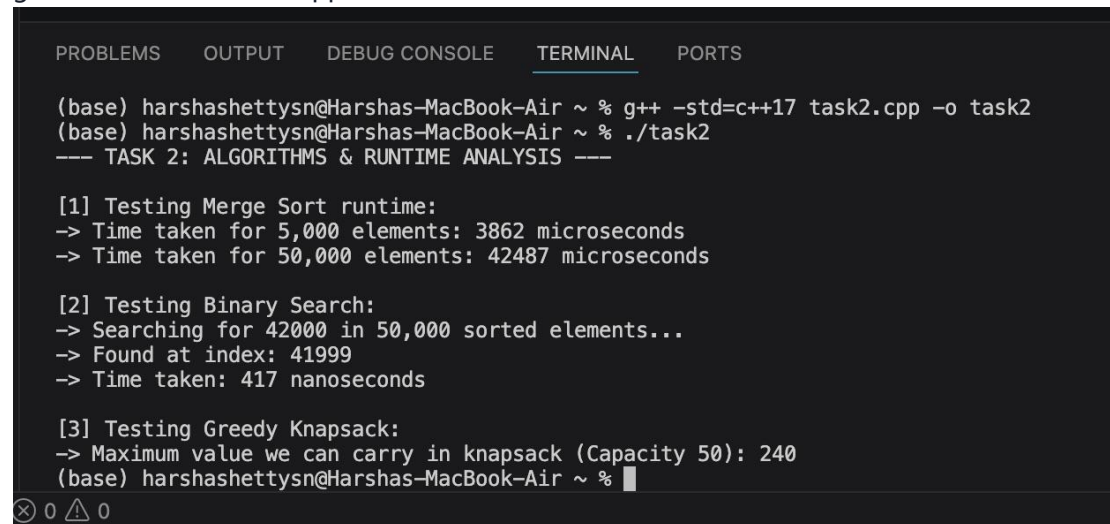
Goal: Implement standard search, sort, and greedy strategy functions alongside explicit code execution

tracking to measure exact CPU runtime overhead variations across varying array densities.

- **Selected Frameworks:** Divide-and-conquer Merge Sort, Binary Search tree paths, and a Greedy fractional knapsack algorithm.

- **Measurement System:** Leveraged the native <chrono> structural library to isolate accurate metrics across small scale inputs versus massive arrays.

```
g++ -std=c++17 task2.cpp -o task2 && ./task2
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) harshashettysn@Harshas-MacBook-Air ~ % g++ -std=c++17 task2.cpp -o task2
(base) harshashettysn@Harshas-MacBook-Air ~ % ./task2
--- TASK 2: ALGORITHMS & RUNTIME ANALYSIS ---

[1] Testing Merge Sort runtime:
-> Time taken for 5,000 elements: 3862 microseconds
-> Time taken for 50,000 elements: 42487 microseconds

[2] Testing Binary Search:
-> Searching for 42000 in 50,000 sorted elements...
-> Found at index: 41999
-> Time taken: 417 nanoseconds

[3] Testing Greedy Knapsack:
-> Maximum value we can carry in knapsack (Capacity 50): 240
(base) harshashettysn@Harshas-MacBook-Air ~ %
```

Task 3: System Programming & File I/O

Goal: Design an atomic system file routine mapping native character reading operations, standard text

streams, error check safety barriers, and token maps.

- **Architecture Details:** Programatically structures an external sample.txt log output block, establishes a robust ifstream parse channel, normalizes character casings, and lists cumulative occurrence rates via a lexical key-sorted map framework.

```
g++ -std=c++17 task3.cpp -o task3 && ./task3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

--- WORD FREQUENCY RESULTS ---

and : 1 times

awesome : 1 times

coding : 2 times

fun : 1 times

internship : 1 times

is : 2 times

learning : 1 times

macbook : 1 times

on : 1 times

programming : 1 times

system : 1 times

to : 2 times

welcome : 2 times

your : 1 times

(base) harshashettysn@Harshas-MacBook-Air ~ %

ⓧ 0 ⚠ 0